CS213 (Autumn 2026) — Quiz 2: TA Grading Rubric

Date: 22 October 2026
Maximum marks: 25
For graders only. This document gives answers and point allocations without reproducing the question statements.

General grading instructions

- Award credit for **semantically equivalent** answers, including different but correct C++ expressions. Do not require exact spelling of the model answer where an equivalent implementation works.
- Grade each code blank independently where possible. For a logically correct solution that distributes operations differently among adjacent blanks, judge the combined code and award the corresponding functional marks.
- In execution traces, allow **error-carried-forward credit** when a student correctly performs the requested next step on their own preceding (incorrect) intermediate array. Do not penalize the same original slip repeatedly. A row copied unchanged earns credit only if no swap or reordering was required in that step (or is a valid carried-forward result).

Marks at a glance

Section	**Maximum**
A. True/False	5
B1. Bubble-sort pass	1.5
B2. LSD radix sort	1.5
C1. Basic counting sort	2
C2. Sorted-list intersection	3
C3. Linear probing with crossing counters	6
C4. Longest common prefixes in a trie	6
Total	**25**

A. True/False — 5 points

0.5 point per correct response; no negative marking.

Item	Key	Brief grading rationale
1	**T**	A max-heap has its maximum at the root/index 0; heap order does not sort the other entries.
2	**F**	Choosing the rightmost maximum does not make swapping-based selection sort stable. Counterexample: `2, 1a, 1b` becomes `1b, 1a, 2` after the first swap.
3	**F**	The cutoff `n/128` grows with the original input size. A constant number (about 128) of insertion-sorted subproblems of size `Theta(n)` can take `Theta(n^2)` total time in the worst case.
4	**T**	An already sorted array with distinct keys can produce a partition of sizes 0 and `n-1` at every step, giving `Theta(n^2)` time.
5	**T**	The zero–one principle applies to any two distinct ordered values, including 2 and 5.
6	**F**	Counting sort takes `Theta(n + k)` time (for the standard implementation), not `Theta(nk)` in general.
7	**T**	An inverted pair after all more-significant digits are processed must agree in all digits except possibly the units digit, so their difference is below 10.
8	**F**	A trie node may represent a proper prefix of stored strings without that prefix itself being stored.
9	**F**	Chaining permits more stored keys than buckets, so load factor `n/m` may exceed 1.
10	**T**	In standard two-choice cuckoo hashing, a stored key must occupy one of its two designated table locations.

Key: `T F F T T F T F F T`

B. Execution — 3 points

B1. Bubble-sort pass — 1.5 points

Step	Correct array	Marks
`i = 0`	`[122, 213, 231, 121, 232]`	0.50
`i = 1`	`[122, 213, 231, 121, 232]`	0.25
`i = 2`	`[122, 213, 121, 231, 232]`	0.50
`i = 3`	`[122, 213, 121, 231, 232]`	0.25
Total		**1.50**

The first and third comparisons require swaps; the second and fourth do not. Use the error-carried-forward rule stated above when appropriate.

B2. LSD radix sort — 1.5 points

Pass	Correct sequence	Marks
Units	`[231, 121, 122, 232, 213]`	0.50
Tens	`[213, 121, 122, 231, 232]`	0.50
Hundreds	`[121, 122, 213, 231, 232]`	0.50
Total		**1.50**

Stability matters when two values have the same digit. Allow error-carried-forward credit for a correctly executed stable digit pass on a preceding incorrect sequence.

C1. Basic counting sort — 2 points

0.5 point per blank, in order:

Blank	Answer	Marks
1	`k`	0.5
2	`x`	0.5
3	`out++`	0.5
4	`v`	0.5
Total		**2**

C2. Intersection of two sorted lists — 3 points

Component	Answer / condition	Marks
First branch	`++i;` or `i++;`	0.5
Second branch	`++j;` or `j++;`	0.5
Equal-keys branch, output	`C.push_back(A[i]);` or `C.push_back(B[j]);`	0.5
Equal-keys branch, advance	Advance **both** `i` and `j` once (`++i; ++j;`)	0.5
Worst-case running time	`O(n+m)`; also accept `Theta(n+m)` for worst-case complexity	1.0
Total		**3.0**

Accept equivalent code that combines appending and incrementing, e.g. `C.push_back(A[i++]);` followed by `++j;`, with full credit for the two equal-branch components. No credit for an equal-branch implementation that emits all copies from one list without respecting minimum multiplicity. The loop advances at least one index per iteration, giving the linear bound.

C3. Linear probing with crossing counters — 6 points

(a) Counter values — 1.5 points

Slot	`crossed`	Marks
3	`1`	0.5
4	`1`	0.5
5	`0`	0.5
Total		**1.5**

After deleting 17, key 24 still crosses both slots 3 and 4 on its way to slot 5. The deleted key 17 no longer contributes to any counter.

(b) Function blanks — 4.5 points

Function	Blank	Expected answer	Marks
`find`	Early termination	`crossed[i] == 0`	1.0
`insert`	Loop condition	`state[i] == OCCUPIED`	1.0
`insert`	Counter update	`++crossed[i]`	1.0
`erase`	Counter update	`--crossed[i]`	1.0
`erase`	Mark deletion	`state[j] = DELETED`	0.5
Total			**4.5**

Acceptable alternatives and cautions:

- For `find`, also accept `state[i] == NEVER_USED || crossed[i] == 0` (the first disjunct is redundant but correct). Merely checking `state[i] == DELETED` or `state[i] != OCCUPIED` is incorrect.
- For the `insert` loop, any logically equivalent test that continues precisely while the slot is occupied is correct. In particular, a `DELETED` slot is reusable; a condition that continues through tombstones is not the intended algorithm.
- For the counter updates, `crossed[i]++` and `crossed[i]--` are equivalent to prefix versions. The updates apply only to slots **before** the final position of the inserted/deleted key, not the final slot itself.
- `state[j] = NEVER_USED` is incorrect for deletion: doing so may invalidate searches for keys farther along the probe sequence.

C4. Longest common prefixes in a trie — 6 points

Part	Expected answer	Marks
(a) `insert` blank	`++p->count` or `p->count++`	1
(a) `erase` blank	`--p->count` or `p->count--`	1
(b)(i)	`k(k-1)/2`	1
(b)(ii)	`v.count * (v.count - 1) / 2`	1.5
(c)	`L_v * v.count * (v.count - 1) / 2`	1.5
Total		**6**

Partial credit and interpretation:

- For (a) insert blank, using `q` instead of `p` is equally correct.
- For (b)(i), alternative `binom(k,2)`. Not dividing by 2 counts ordered pairs and earns **0.5/1**; `k` or `k*k` earns 0.
- For (b)(ii), alternative `binom(v.count,2)`. Award **1/1.5** if `/2` omitted; otherwise 0.
- For (c), alternative `L_v * binom(v.count,2)`. Award **0.5/1.5** for omitting the edge-length factor `L_v`, or for the correct `L_v` factor attached to a wrong pair-count expression like `v.count` or `v.count*v.count`. Award **1/1.5** for omitting `/2`.

The underlying invariant is that each unordered pair contributes one unit for each character of its shared prefix. In a radix tree, an edge of label length `L_v` contributes that many units per pair sharing the prefix represented by its child.
